ragent Toolchain Report Manual

This guide explains ragent's `/toolchain` slash command — a read-only runtime-toolchain presence report that shows which language toolchains are installed on the local machine, at what versions, for every language the code index can parse. It covers the command surface, the language classification, the runtime mapping, the probe mechanics, and the report formats.

> **Scope:** the `/toolchain` slash command (TUI-only), the `crates/ragent-tui/src/app/toolchain.rs` probe engine, and the language-to-runtime mapping. For the code index itself see `docs/howtos/codeindex.md`. For the full tool catalog see `docs/howtos/tools/INDEX.md`. For TUI workflow see `docs/howtos/tutorial.md`.

Table of Contents

1. Overview
2. Command Reference
3. Language Classification
4. Runtime Mapping
5. Probing Mechanics
6. Report Output
7. JSON Output
8. Non-Blocking Execution
9. Design Guarantees
10. Implementation Map

---

1. Overview

The code index (`ragent-codeindex::scanner`) maintains `SUPPORTED_LANGUAGES`, a 50-entry list of language identifiers it can parse (50 canonical entries including dialect/header aliases such as `tsx`, `jsx`, `c_header`, `cpp_header`). Those entries split into two groups:

- **Application languages** (30 entries) — languages whose source is compiled or interpreted by a runtime toolchain (`rust` needs `cargo`/`rustc`, `python` needs `python3`, `go` needs `go`, and so on).
- **Data/format entries** (20 entries) — pure data, markup, or build-DSL formats with no source runtime (`toml`, `yaml`, `json`, `xml`, `html`, `css`, `scss`, `sql`, `markdown`, `protobuf`, `verilog`, `vhdl`, `terraform`, `openscad`, `cmake`, `gradle`, `gradle_kts`, `maven`, `nix`, `hcl`).

There was no way, from inside ragent, to see which of those runtimes are actually installed locally and at what version. `/toolchain` fills that gap: it walks `SUPPORTED_LANGUAGES`, classifies every entry, probes the local system for each application language's runtime, and reports presence and version in a fixed-width ASCII table or a JSON document.

Key properties:

- **Read-only** — no installs, upgrades, configuration changes, or state mutation. The only child processes are version probes.
- **Derived, not duplicated** — the language set is derived from `SUPPORTED_LANGUAGES` at runtime; the mapping table is exhaustiveness- checked against it, so the two can never drift silently.

- **Non-blocking** — probes run on a blocking thread with a `[wait] toolchain` status indicator, so the TUI stays responsive.
- **Fault-tolerant** — a missing or misbehaving runtime never aborts the report; each probe is bounded by a 2-second timeout.

---

2. Command Reference

Command	Description
`/toolchain`	Show the help page (same as `help`)
`/toolchain help`	Help page with subcommand table and worked example
`/toolchain list`	Full report: every supported language, its runtime(s), presence, and version
`/toolchain list <language>`	Report filtered to one language id (matched case-insensitively)
`/toolchain list --json`	Machine-readable JSON rendering of the report

Argument notes:

- The `--json` flag is position-independent and combinable with a language filter (`/toolchain list rust --json`).
- An unknown subcommand (`/toolchain frobnicate`) renders a usage correction and dispatches no probe.
- An unknown language id (`/toolchain list nosuchlang`) warns, echoes the valid id list, and produces no report table.

The status bar reflects the surface: `toolchain: help`, `toolchain: list`, `toolchain: usage`, or `[wait] toolchain` while the probes run.

Help page

`/toolchain help` (and bare `/toolchain`) renders:

```
From: /toolchain help

## /toolchain - Runtime Toolchain Presence Report

Reports which language toolchains are installed on this machine,
walking the code-index supported-language list. The report is
**read-only** - it never installs, upgrades, or configures any
runtime, and it never touches ragent state.

| Subcommand | Description |
|---|---|
| `/toolchain help` | Show this help page |
| `/toolchain list` | Full report: every supported language, its runtime(s), presence, and versio
| `/toolchain list <language>` | Report filtered to one language id (matched case-insensitively)
```

| `/toolchain list --json` | Machine-readable JSON rendering of the report |

Example: `/toolchain list rust` shows the rust row (probes `cargo` and `rustc`).

---

3. Language Classification

Every SUPPORTED_LANGUAGES entry is classified via a single mapping table (LANGUAGE_RUNTIMES) as either an application language or a data/format entry. The table covers all 50 canonical ids exactly once; an exhaustiveness check (exhaustiveness_errors()) is enforced by tests, so adding a language to the scanner const without adding a mapping row is a detectable gap.

Application (30): rust, python, typescript, tsx, javascript, jsx, go, c, c_header, cpp, cpp_header, java, kotlin, ruby, swift, csharp, lua, shell, zsh, fish, zig, nim, elixir, erlang, haskell, ocaml, r, dart, php, perl.

Data/format (20): toml, yaml, json, xml, html, css, scss, sql, markdown, protobuf, verilog, vhdl, terraform, openscad, cmake, gradle, gradle_kts, maven, nix, hcl.

Data-format rows still appear in the full list report, marked (data format - runtime n/a), so the user sees the full language surface; only application rows carry presence/version columns.

---

4. Runtime Mapping

Each application language maps to the runtime command(s) required to build and run code in it. The mapping lives in LANGUAGE_RUNTIMES as (id, class, runtime) rows:

Language id(s)	Runtime probe(s)
rust	cargo, rustc
python	python3
typescript, tsx	node, tsc
javascript, jsx	node
go	go
c, c_header	cc (fallback gcc, then clang)
cpp, cpp_header	c++ (fallback g++, then clang++)
java	java, javac
kotlin	kotlinc
ruby	ruby
swift	swift
csharp	dotnet
lua	lua (fallback luajit)
shell	bash
zsh	zsh
fish	fish
zig	zig
nim	nim
elixir	elixir
erlang	erl

Language id(s)	Runtime probe(s)
haskell	ghc (fallback runghc)
ocaml	ocaml
r	R
dart	dart
php	php
perl	perl

Multi-runtime rows report the per-command status: `cargo: installed; rustc: installed`. A language row counts as installed when the primary probe (or a declared fallback) resolves on `PATH`.

---

5. Probing Mechanics

Presence probe

Presence is a pure `PATH` resolution (`command -v` semantics) — the command name is resolved against the `PATH` environment variable without spawning the tool. On Unix the candidate must exist and carry an execute bit; on Windows the file must exist and match a `PATHEXT` extension (`.COM`, `.EXE`, `.BAT` by default). No network access is involved.

Version probe

For each installed runtime, the engine executes `<command> <version-flag>` and captures the first non-empty line from stdout or stderr, trimmed and rendered on a single line. The default flag is `--version`; exceptions are held in `VERSION_FLAG_OVERRIDES`:

Command	Version flag
java	-version (reports on stderr)
erl	-version (reports on stderr)
lua	-v
luajit	-v
go	version (subcommand)
zig	version (subcommand)

`dotnet` is special-cased in `VERSION_PROBE_ARGS_OVERRIDES`: `dotnet --version` prints only the active SDK version (and fails on runtime-only installs), so the probe combines `--list-sdks` and `--list-runtimes` and joins the first line of each into one `,`-separated version string.

Failure containment

Every version probe is bounded by a 2-second timeout (`VERSION_PROBE_TIMEOUT`). A probe that fails to start, exits non-zero without usable version text, or exceeds the timeout reports version `unknown` or `timeout` respectively — while the runtime is still reported `installed` — and the report continues. A missing runtime costs one `PATH` walk each (no spawn) and never aborts or truncates the report.

No shell injection

All spawned commands are compile-time constants from the runtime mapping. User input is used only for the case-insensitive language-filter match and is never interpolated into a spawned command line.

---

6. Report Output

`/toolchain list` renders a fixed-width ASCII table prefixed `From: /toolchain list` with columns **Language**, **Runtime**, **Status**, **Version** fixed at 10, 10, 10, and 50 characters respectively (FR-017; Language and Runtime cells clip to the column width, the Status and Version columns word-wrap onto continuation grid lines) — followed by a summary line:

```
From: /toolchain list

+------------+------------+------------+--------------------------------------------------+
| Language   | Runtime    | Status     | Version                                          |
+------------+------------+------------+--------------------------------------------------+
| rust       | cargo, rus | cargo:     | cargo: rustc 1.98.1; rustc: rustc 1.98.1         |
+------------+------------+------------+--------------------------------------------------+
|            |            | installed; |                                                  |
+------------+------------+------------+--------------------------------------------------+
|            |            | rustc:     |                                                  |
+------------+------------+------------+--------------------------------------------------+
|            |            | installed  |                                                  |
+------------+------------+------------+--------------------------------------------------+
| python     | python3    | installed  | Python 3.12.5                                    |
+------------+------------+------------+--------------------------------------------------+
| go         | go         | installed  | go version go1.24.5 linux/amd64                  |
+------------+------------+------------+--------------------------------------------------+
| toml       | -          | (data      | -                                                |
+------------+------------+------------+--------------------------------------------------+
|            |            | format -   |                                                  |
+------------+------------+------------+--------------------------------------------------+
|            |            | runtime    |                                                  |
+------------+------------+------------+--------------------------------------------------+
|            |            | n/a)       |                                                  |
+------------+------------+------------+--------------------------------------------------+
| erlang     | erl        | not        | -                                                |
+------------+------------+------------+--------------------------------------------------+
|            |            | installed  |                                                  |
+------------+------------+------------+--------------------------------------------------+
...
12/30 application runtimes installed.
```

- Rows follow `SUPPORTED_LANGUAGES` order; repeated invocations in the same environment produce identical columns (barring concurrent `PATH` changes).
- Multi-runtime rows join per-command status with ; and per-command version with ; (the Status and Version columns word-wrap onto continuation lines when longer, with blank Language/Runtime cells — and a blank Status cell on Version-only continuation lines).

- Absent runtimes render `not installed` with a `-` version placeholder.
- The summary line counts application rows with at least one installed runtime over the number of application rows.

---

7. JSON Output

`/toolchain list --json` emits a bare JSON document with no `From:` prefix, so it can be piped or parsed directly:

```
{
  "languages": [
    {
      "id": "rust",
      "runtime": "cargo, rustc",
      "status": "cargo: installed; rustc: installed",
      "version": "cargo: rustc 1.98.1; rustc: rustc 1.98.1"
    },
    {
      "id": "toml",
      "runtime": "-",
      "status": "(data format - runtime n/a)",
      "version": "-"
    }
  ],
  "installed": 12,
  "total": 30
}
```

The `installed`/`total` counters use the same application-row logic as the markdown summary line. `--json` is combinable with a language filter.

---

8. Non-Blocking Execution

The TUI is a single-threaded UI loop; spawning dozens of child processes inline would stall it. `/toolchain list` therefore runs the full walk and probe pass on a blocking thread (`tokio::task::block_in_place` over `build_report_rows`) with the status bar set to `[wait] toolchain` until the report is ready. The same `[wait]` status prefix is the hook the TUI uses to defer event handling while an async slash command runs (shared with `/bench` and `/swarm`).

On a typical Linux workstation a full report completes within 10 seconds — absent runtimes cost one `PATH` walk each, and present runtimes are bounded by the 2-second per-probe timeout.

---

9. Design Guarantees

- **Exhaustive mapping** — `exhaustiveness_errors()` verifies every `SUPPORTED_LANGUAGES` entry has exactly one `LANGUAGE_RUNTIMES` row (and vice versa); the check runs in the module's test

suite, so drift between the scanner list and the mapping is caught in CI, not in production.

- **No second language list** — the report derives its language set from the scanner const at runtime; there is no hard-coded duplicate to drift.
- **Read-only** — no filesystem writes, no configuration mutation, no installers, no code-index changes. The only child processes are version probes and `PATH` resolution.
- **Fail-open reporting** — missing, failing, or slow runtimes degrade to `not installed` / `unknown` / `timeout` placeholders rather than aborting the report.
- **TUI-safe** — blocking probes run off the UI loop with a `[wait]` indicator.

---

10. Implementation Map

Component	Location
Language classification + runtime mapping	`crates/ragent-tui/src/app/toolchain.rs` (`LANGUAGE_RUNTIMES`, `LanguageClass`, `LanguageRuntime`)
Exhaustiveness check	`toolchain.rs` (`exhaustiveness_errors`)
Presence probe (`PATH` walk)	`toolchain.rs` (`probe_installed`, `resolve_in_dirs`)
Version probes + overrides	`toolchain.rs` (`probe_version`, `VERSION_FLAG_OVERRIDES`, `VERSION_PROBE_ARGS_OVERRIDES`, `VERSION_PROBE_TIMEOUT`)
Report walk	`toolchain.rs` (`build_report_rows`)
Markdown / JSON renderers	`toolchain.rs` (`render_markdown_report`, `render_json_report`)
Slash dispatch + help/usage/filter/`--json` handling	`crates/ragent-tui/src/app/slash.rs` (`handle_toolchain_command`, `handle_toolchain_help`, `handle_toolchain_list`, `handle_toolchain_unknown`)
Command registration + autocomplete	`crates/ragent-tui/src/app/state.rs` (`SLASH_COMMANDS`), `slash.rs` (`get_command_suggestions`)
Spec (requirements FR-001 to FR-016, NFR-001 to NFR-003)	`specs/toolchain/SPEC.md`
Manual test-plan driver	`crates/ragent-tui/examples/toolchain_tc_pass.rs`